Maverick Feedback 360 — Full Project Details

This document explains the Mavericks Feedback 360 project in full detail for the Designathon expo. It covers the product vision, architectural design, data model, user workflows, backend routes, frontend pages, security, analytics, and the exact demo narrative.

Project Vision

Maverick Feedback 360 is designed to make training feedback collection, reporting, and evaluation seamless across Admin, Trainer, Supervisor, and Maverick roles.

The goal is to replace manual Excel and email-based feedback processes with a centralized, auditable, and analytics-ready system.

Target Users and Use Cases

- Admins who create training courses, assign trainers, manage sessions, and oversee feedback completion.
- Mavericks (trainees) who complete feedback forms after attending sessions and maintain training history.
- Supervisors who evaluate Mavericks and provide structured performance reviews.
- Organizations that need traceability, audit logging, and actionable analytics from training feedback.

High-Level Architecture

The application is divided into a backend REST API and a frontend Single Page Application.

Backend responsibilities include data access, authentication, authorization, feedback cycle processing, notifications, and analytic endpoints.

Frontend responsibilities include role-based dashboards, protected navigation, form workflows, and real-time feedback for users.

Technology Stack

- Backend: Node.js, Express, Prisma ORM, SQLite, JWT, bcryptjs, multer, xlsx, nodemailer stub, dotenv, cors.
- Frontend: React 19, Vite, React Router DOM, Axios, Tailwind CSS, Lucide icons, React Hot Toast, Recharts.

Backend Architecture

The backend entry point is backend/src/server.js. It initializes Express, enables JSON body parsing, configures CORS, mounts routes, and handles global errors.

Each feature area has its own route file: auth, users, master data, participants, feedback, analytics, notifications, and audit logs.

Common logic is separated into middleware, utility, and service files for maintainability.

Authentication and Authorization

Authentication depends on JWT tokens. The middleware reads the Bearer token, validates it against the server secret, and attaches the decoded user to the request.

Authorization is enforced by role: Admin-only endpoints use middleware that fails requests from unauthorized roles. The same middleware prevents Mavericks and Supervisors from accessing admin-only workflows.

Key Backend Routes and Behavior

- POST /api/auth/login: accepts email and password, returns accessToken and expiresIn.
- POST /api/auth/otp/request: generates a temporary OTP, stores it, and logs it for demo purposes.
- POST /api/auth/otp/verify: validates OTP and returns a JWT token.
- GET /api/auth/me: returns the current authenticated user profile.
- GET /api/users: Admin-only user listing with pagination and filters.
- POST /api/users: Admin can add a new user with hashed password initialization.
- PUT /api/users/:id: Admin edits user details.
- PATCH /api/users/:id/status: Admin activates/deactivates accounts.
- GET /api/users/:id/supervisor-mapping: fetches supervisor assignment info.
- PUT /api/users/:id/supervisor-mapping: sets the supervisor for a Maverick.

Master Data and Session Management

Admin can manage training courses, trainers, and sessions through master endpoints.

Session creation includes validation and a trainer schedule conflict check using overlapping date logic.

Courses and trainers can be filtered by status, type, domain, or engagement type.

Participants and Enrollment

Participants are uploaded as an Excel spreadsheet using a defined template. The backend validates each row, matches employee IDs to existing users, assigns supervisors, and creates enrollment records.

Enrollment triggers notifications to participants, demonstrating a real notification workflow in the demo.

Feedback Cycles

Feedback cycles are the central process that binds sessions to feedback forms.

Each cycle can be Maverick feedback or Supervisor evaluation, and it tracks completion thresholds.

When enough forms are submitted, the cycle automatically closes and analytics are triggered.

Admins can also force-close a cycle with a reason, which is recorded for compliance.

Maverick Feedback Forms

Maverick forms support drafts, autosave, and final submission. Each form tracks overall rating, learnings, improvements, and follow-up responses.

Draft saving uses a PUT endpoint and is triggered manually by the user and automatically every minute in the frontend.

Supervisor Evaluation Forms

Supervisor forms capture five performance criteria plus manager comments. Evaluations also support drafts and final submission.

When a supervisor submits an evaluation, the backend computes an average and sends alerts if performance is below thresholds.

Analytics and Intelligence

Analytics endpoints power dashboard metrics, session scorecards, trainer performance, and leaderboards.

The backend includes simple AI modules that simulate sentiment analysis and theme clustering for Maverick feedback text.

These features demonstrate how the product can evolve into a richer learning analytics platform.

Notification and Audit

Notifications are stored in the database and displayed in user dashboards.

Users can mark notifications read individually or all at once.

Audit logs capture all Admin actions with actor, action type, entity type, old values, and new values.

The audit log supports query filters so admin activity can be reviewed in the expo.

Database Model

The data model is implemented in Prisma and stored in SQLite for a portable demo environment.

User records support role-based fields and supervisor relationships.

Sessions connect courses, trainers, admins, participants, and feedback cycles.

Feedback forms connect Mavericks and Supervisors to cycles, enabling complete traceability of feedback history.

Key Tables and Entities

- User: stores identity, role, status, supervisor relationships, and authentication data.
- Course: stores training course details, type, objectives, and duration.
- Trainer: stores trainer profile, domain, and engagement type.
- Session: stores scheduled training sessions and connects course, trainer, and admin.
- FeedbackCycle: stores cycle type, status, threshold, deadlines, and closure state.
- MaverickFeedbackForm / SupervisorEvaluationForm: store feedback/questionnaire data and submission status.

Frontend Architecture

The React frontend is designed for fast iteration with Vite and modular route-based views.

Authentication state is held in context and synchronized with localStorage so users remain signed in on refresh.

The layout adapts to the signed-in role, showing only the navigation items relevant to that user.

Core Frontend Pages

- Login: supports email/password and OTP verification paths.
- Admin Dashboard: overview cards, recent sessions, and alerts.
- Maverick Dashboard: pending feedback, submissions, and notifications.
- Supervisor Dashboard: pending evaluations, completed reviews, and notifications.
- Sessions/Courses/Trainers pages: data tables with actions and status filters.
- Feedback and evaluation forms: interactive star ratings, text inputs, and submission controls.

User Experience Details

The UI uses cards, badges, tables, and consistent headers to provide a polished experience.

Form pages include validation messaging, read-only mode after submission, and contextual summaries.

Dashboards include quick counts and direct links to workflows, making it easy for users to complete actions.

Detailed Demo Plan

Start with the application vision and architecture overview.

Show the Admin dashboard and explain how it supports training operations.

Create or edit master data to prove the app can manage courses, trainers, and sessions.

Upload participants via Excel to show bulk enrollment and supervision mapping.

Switch to Maverick and demonstrate draft saving plus submission of feedback.

Then switch to Supervisor and demonstrate evaluation, scoring, and comments.

Finish with analytics, leaderboards, notifications, and audit logs to prove the system supports reporting and governance.

Setup and Execution

- Install backend dependencies: cd backend && npm install.
- Install frontend dependencies: cd frontend && npm install.
- Run the backend: cd backend && npm run dev.
- Run the frontend: cd frontend && npm run dev.
- Seed the database if needed: cd backend && npm run seed.

Future Enhancements

Move the sentiment/theme analysis stub to a real NLP service like AWS Comprehend, Azure Cognitive Services, or Hugging Face.

Add richer Recharts visualizations for feedback trends and training performance.

Implement real email/SMS delivery with SendGrid, SES, or Twilio.

Add auditing and security improvements such as refresh tokens, role-based UI restrictions, and stricter input validation.

Create end-to-end tests for backend APIs and frontend pages.

Why This Project Wins

It solves a real training logistics problem by centralizing feedback, evaluation, and analytics.

It is designed for rapid demo deployment with a portable SQLite database and reusable Excel import templates.

It supports enterprise-ready governance with audit logs and role-based security.

This PDF is intended to support a strong Designathon expo presentation by giving you the exact architecture, feature list, demo narrative, and technical depth needed to explain the project clearly.